Desenvolvimento de Smart Contract com Registro e Recompensa

Carlos Eduardo Alves da Silva

Git do projeto: https://github.com/Eduprogrammer/RegistroDeUsuariosComRecompensa

Objetivo

Desenvolvimento de um Smart Contract em Solidity para registro de usuários e distribuição de recompensas em tokens simulados, aplicando conceitos fundamentais de blockchain e Ethereum Virtual Machine (EVM).

Introdução

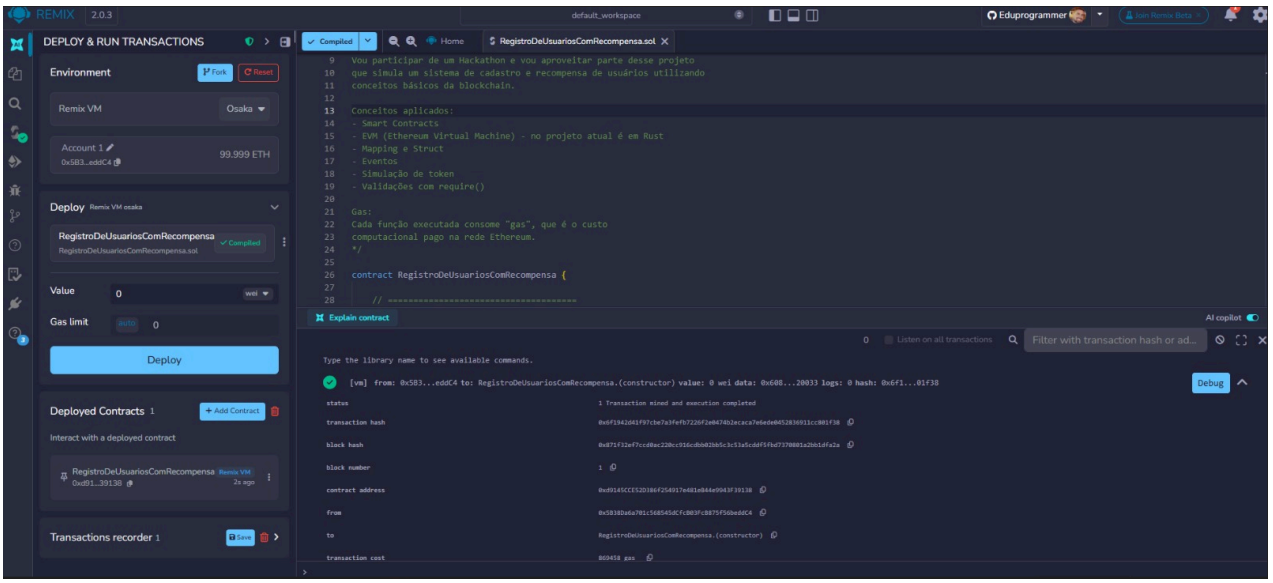
Vou participar de um hackathon da blockchain Stellar e aproveitei uma parte do projeto para demonstrar registros de usuários e distribuição de recompensas em tokens simulados na linguagem Solidity. O contrato utiliza struct, mapping, eventos e validações com require(), garantindo integridade e segurança das operações.

As funcionalidades foram testadas no Remix IDE, comprovando o correto funcionamento do sistema, incluindo emissão de eventos e controle de duplicidade.

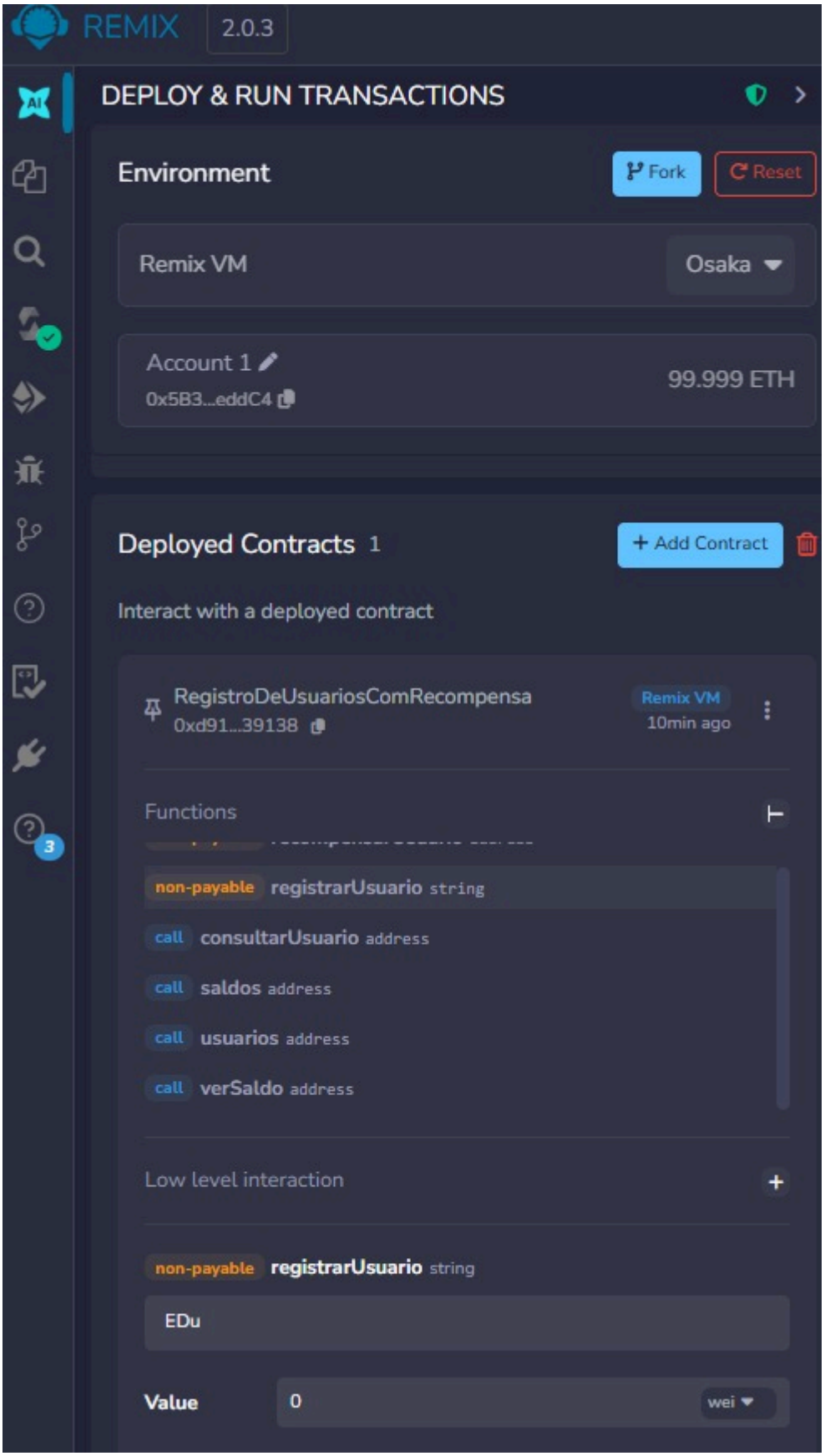
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;


contract RegistroDeUsuariosComRecompensa {

    // ====================================
    // STRUCT (estrutura do usuário)
    // ====================================
    struct Usuario {
        string nome;
        bool cadastrado;
        bool recebeuRecompensa;
    }

    // ====================================
    // STORAGE (armazenamento na blockchain)
    // ====================================

    // Mapeia endereço para dados do usuário
    mapping(address => Usuario) public usuarios;

    // Saldo dos tokens
    mapping(address => uint) public saldos;

    // ====================================
    // EVENTOS (logs registrados na blockchain)
    // ====================================

    event UsuarioRegistrado(address carteira, string nome);
    event RecompensaEnviada(address carteira, uint valor);

    // ====================================
    // FUNÇÃO: Registrar usuário
    // ====================================
    function registrarUsuario(string memory nome) public {
```

```
        // Evita cadastro duplicado
        require(!usuarios[msg.sender].cadastrado, "Usuario ja registrado");

        usuarios[msg.sender] = Usuario(nome, true, false);

        emit UsuarioRegistrado(msg.sender, nome);
    }

    // ======================================
    // FUNÇÃO: Consultar usuário
    // ======================================
    function consultarUsuario(address carteira) public view returns (string memory, bool, bool) {

        Usuario memory u = usuarios[carteira];

        return (u.nome, u.cadastrado, u.recebeuRecompensa);
    }

    // ======================================
    // FUNÇÃO: Recompensar usuário
    // ======================================
    function recompensarUsuario(address carteira) public {

        Usuario storage u = usuarios[carteira];

        // Verifica se está cadastrado
        require(u.cadastrado, "Usuario nao cadastrado");

        // Evita recompensa duplicada
        require(!u.recebeuRecompensa, "Usuario ja recompensado");

        uint recompensa = 100;

        saldos[carteira] += recompensa;
        u.recebeuRecompensa = true;

        emit RecompensaEnviada(carteira, recompensa);
    }

    // ======================================
    // FUNÇÃO: Ver saldo
    // ======================================
    function verSaldo(address carteira) public view returns (uint) {
        return saldos[carteira];
    }


    /*
    ==========================================================
    EXPLICAÇÃO FINAL
    ==========================================================

    Este contrato é executado na EVM

    ==========================================================
    CASO DE USO REAL
    ==========================================================

    Este contrato pode ser aplicado em:

    - Academias: usuários recebem recompensas ao cumprir desafios
    - Apps de produtividade: recompensa por tarefas concluídas
    - Plataformas educacionais: incentivo ao aprendizado
    - Sistemas Web3 de gamificação

    Exemplo (Gym):
    No contrato que eu vou participar do Hackathon o usuário se cadastra, participa de um desafio na academia e ao cumprir
    a meta recebe tokens como recompensa automática.
    */
}
```

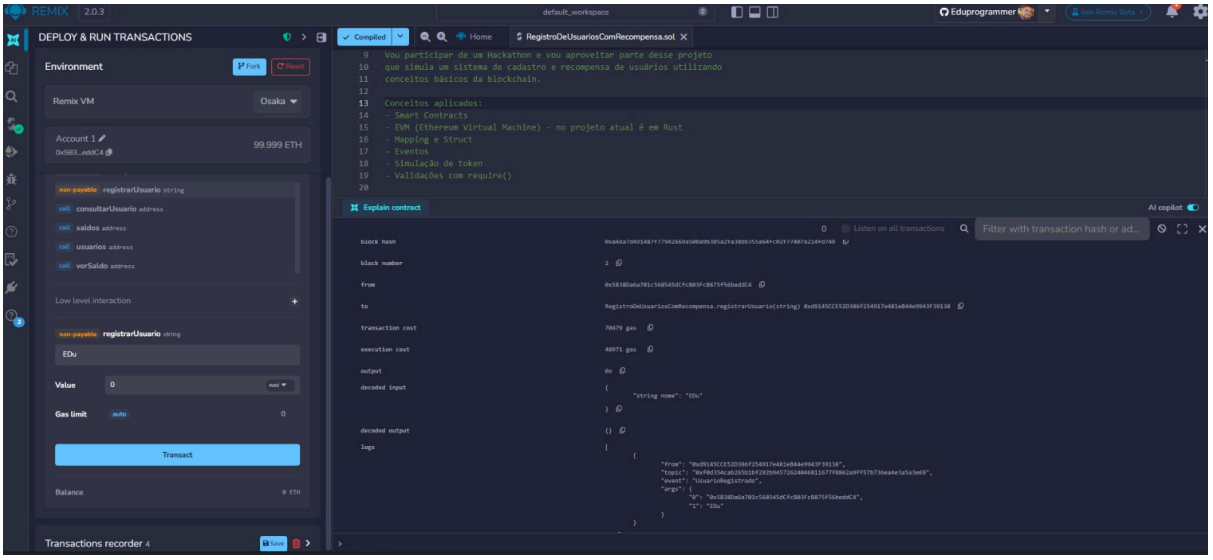
Deploy do Smart Contract na rede simulada (Remix VM)



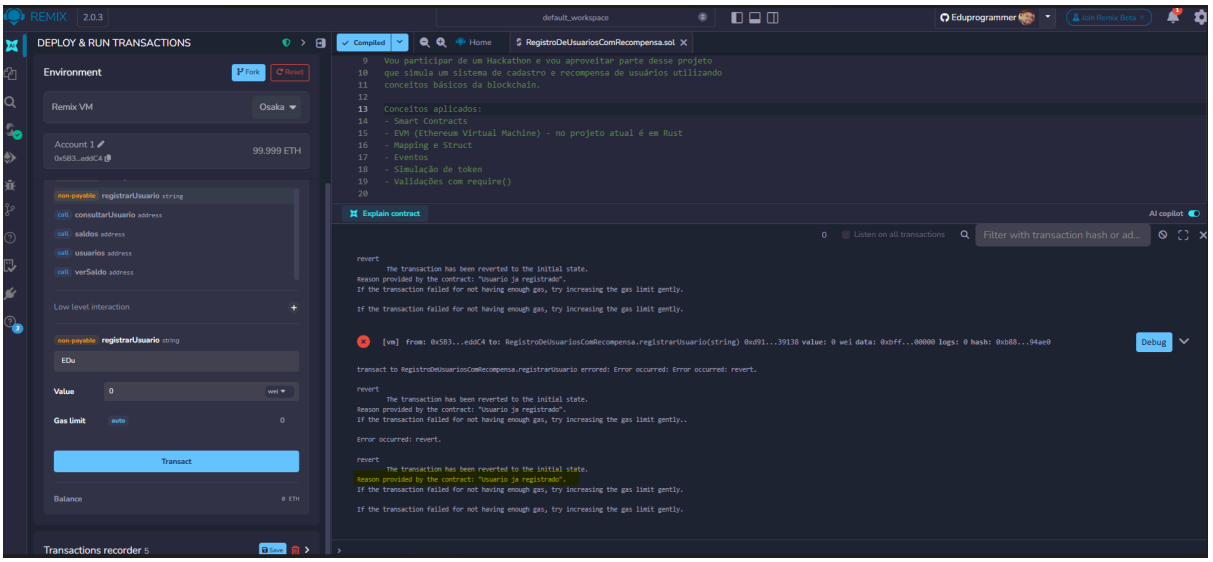
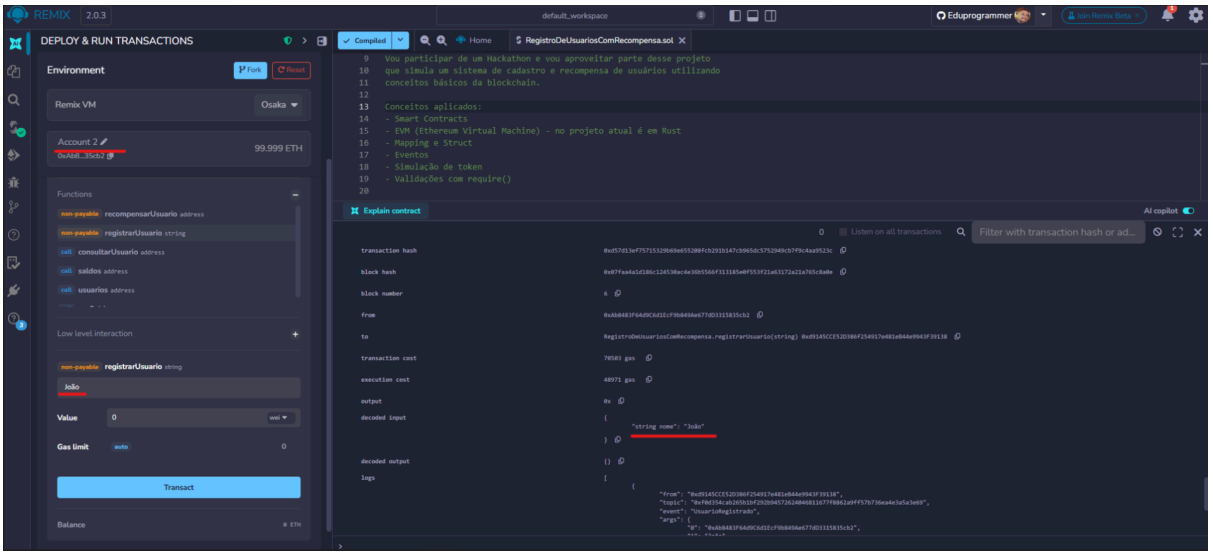
Execução da função de registro de usuário e emissão de evento



decoded input { "string nome": "EDu"}



Validação de segurança: tentativa de registro duplicado bloqueada



Registro de novo usuário com sucesso e armazenamento em blockchain